

Prim's Algorithm - Code, Output and Time Complexity

Objective: To find the Minimum Spanning Tree (MST) of a weighted connected graph using Prim's algorithm and understand the time complexity of different implementations.

1. Method Used in the Given C Program

The given program implements Prim's algorithm using an **adjacency matrix**. It begins with vertex 0 and uses the **selected[]** array to keep track of vertices already included in the MST. At each step, it searches for the minimum-weight edge that connects a selected vertex to an unselected vertex. The process continues until **V - 1** edges are selected. With this matrix-based scanning approach, the time complexity is approximately **O(V²)**.

2. C Code - Prim's Algorithm

```
#include <stdio.h>

#define INF 9999
#define V 5

void prim(int graph[V][V])
{
    int selected[V] = {0};
    int edges = 0;
    int totalCost = 0;

    selected[0] = 1;

    printf("Edges in Minimum Spanning Tree:\n");

    while (edges < V - 1)
    {
        int min = INF;
        int x = 0, y = 0;

        for (int i = 0; i < V; i++)
        {
            if (selected[i])
            {
                for (int j = 0; j < V; j++)
                {
                    if (!selected[j] && graph[i][j] != 0)
                    {
                        if (graph[i][j] < min)
                        {
                            min = graph[i][j];
                            x = i;
                            y = j;
                        }
                    }
                }
            }
        }

        printf("%d - %d : %d\n", x, y, graph[x][y]);
        totalCost += graph[x][y];
        selected[y] = 1;
        edges++;
    }

    printf("Total cost of MST = %d\n", totalCost);
}

int main()
{
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };

    prim(graph);

    return 0;
}
```

3. Program Output

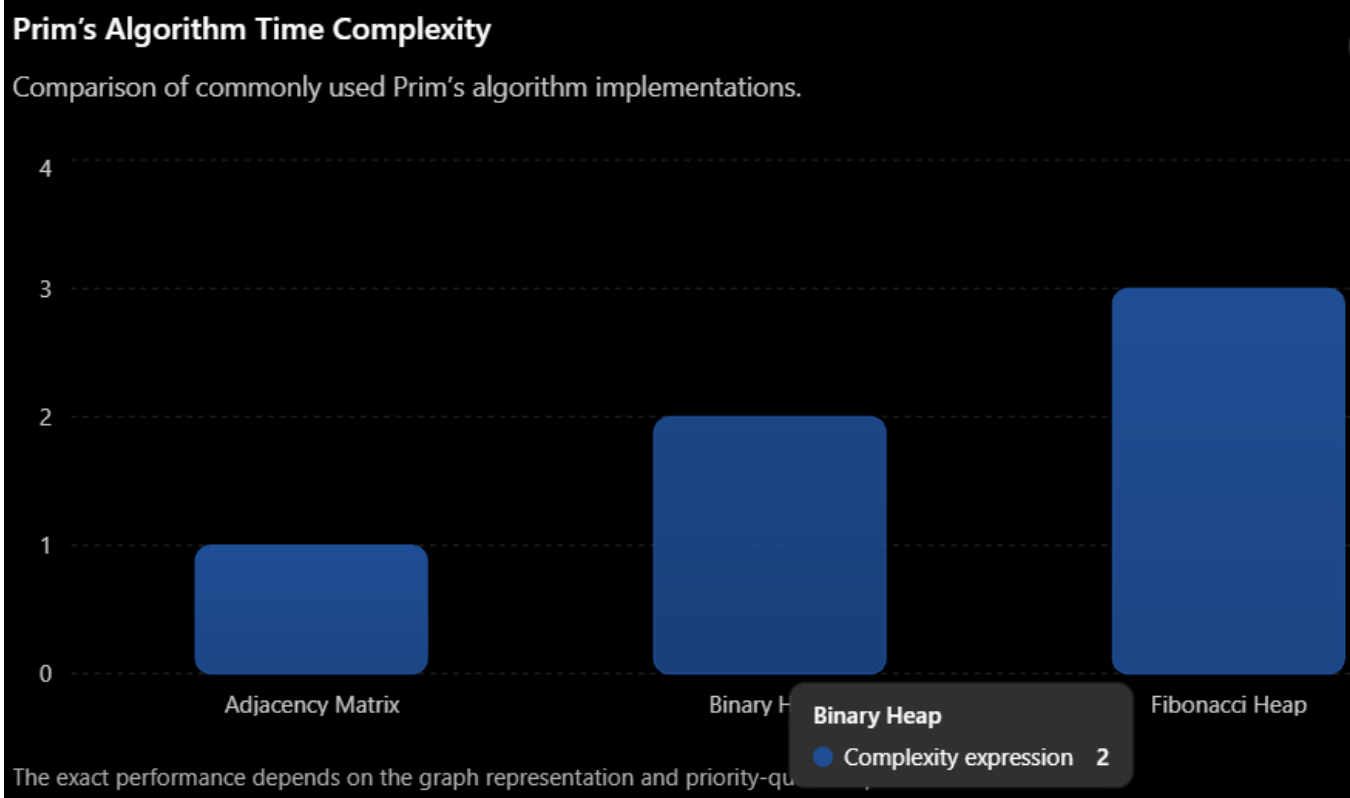
```
[Running] cd "c:\Users\Aadhya Jindal\OneDrive\Desktop\DAA Aadhya Jindal\" && gcc prims.c -o prims && "c:\Users\Aadhya Jindal\OneDrive\Desktop\DAA Aadhya Jindal\prims
Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Total cost of MST = 16
[Done] exited with code=0 in 4.177 seconds
```

The program selects the MST edges **0-1 (2)**, **1-2 (3)**, **1-4 (5)**, and **0-3 (6)**. Their total weight is $2 + 3 + 5 + 6 = 16$, so the total cost of the Minimum Spanning Tree is **16**.

4. Alternative Implementations

Prim's algorithm can also be implemented using an **adjacency list with a priority queue**. With a **binary heap**, the commonly stated complexity is $O(E \log V)$, which is useful for sparse graphs. A **Fibonacci heap** gives the theoretical bound $O(E + V \log V)$, but it is more complicated to implement than a binary heap.

5. Time Complexity Chart



The chart compares three common Prim's algorithm implementations. The **adjacency matrix** method is straightforward and has $O(V^2)$ complexity. A **binary heap** with an adjacency list reduces the work to about $O(E \log V)$. A **Fibonacci heap** provides the best theoretical bound of $O(E + V \log V)$, although its implementation is more complex. The chart is a conceptual comparison of complexity expressions, not measured execution time.